

🔥 Social Fit — Firebase Implementation Rehberi

React Native + Expo | Auth + Firestore + Storage

📦 1. KURULUM

1.1 Firebase SDK'yı Yükle

```
bash

npx expo install firebase
npx expo install @react-native-async-storage/async-storage
npx expo install expo-auth-session expo-crypto expo-web-browser
```

1.2 Google & Apple Auth için Native Modüller

```
bash

# Google Sign-In
npx expo install @react-native-google-signin/google-signin

# Apple Sign-In (sadece iOS)
npx expo install expo-apple-authentication
```

1.3 app.json / app.config.js Güncelle

```
json

{
  "expo": {
    "plugins": [
      "@react-native-google-signin/google-signin",
      "expo-apple-authentication"
    ],
    "ios": {
      "bundleIdentifier": "com.socialfit.app",
      "usesAppleSignIn": true
    },
    "android": {
      "package": "com.socialfit.app",
      "googleServicesFile": "./google-services.json"
    }
  }
}
```

⚠ Expo Go ile Google/Apple Auth **çalışmaz**. `eas build --profile development` ile development build kullanman gerekir.

⚙ 2. FIREBASE YAPILANDIRMASI

2.1 Firebase Console Adımları

1. console.firebase.google.com → Yeni proje oluştur: "social-fit"
2. **Authentication** → Sign-in methods → Google, Apple, Email/Password'u aç
3. **Firestore Database** → Create database → Production mode
4. **Storage** → Get started
5. iOS için `GoogleService-Info.plist`, Android için `google-services.json` indir → proje köküne koy

2.2 Firebase Config Dosyası

typescript

```
// lib/firebase/config.ts
import { initializeApp, getApps } from 'firebase/app';
import { getAuth, initializeAuth, getReactNativePersistence } from 'firebase/auth';
import { getFirestore } from 'firebase/firestore';
import { getStorage } from 'firebase/storage';
import AsyncStorage from '@react-native-async-storage/async-storage';

const firebaseConfig = {
  apiKey: process.env.EXPO_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.EXPO_PUBLIC_FIREBASE_PROJECT_ID,
  storageBucket: process.env.EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.EXPO_PUBLIC_FIREBASE_APP_ID,
};

// Çift init önleme
const app = getApps().length === 0 ? initializeApp(firebaseConfig) : getApps()[0];

// Auth - AsyncStorage ile persist
export const auth = initializeAuth(app, {
  persistence: getReactNativePersistence(AsyncStorage),
});

export const db = getFirestore(app);
export const storage = getStorage(app);
```

2.3 .env Dosyası

bash

```
# .env.local (git'e commit etme!)
EXPO_PUBLIC_FIREBASE_API_KEY=AIza...
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=social-fit.firebaseio.com
EXPO_PUBLIC_FIREBASE_PROJECT_ID=social-fit
EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET=social-fit.appspot.com
EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=123456789
EXPO_PUBLIC_FIREBASE_APP_ID=1:123:web:abc
```

3. AUTH SERVİSİ

3.1 Auth Hook (useAuth)

typescript

```
// hooks/useAuth.ts
import { useState, useEffect } from 'react';
import { auth } from '@lib/firebase/config';
import { onAuthStateChanged, User } from 'firebase/auth';

export function useAuth() {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, (user) => {
      setUser(user);
      setLoading(false);
    });
    return unsubscribe;
  }, []);

  return { user, loading };
}
```

3.2 Email / Password Auth

typescript

```
// lib/firebase/auth.ts
import {
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  signOut,
  sendPasswordResetEmail,
  updateProfile,
} from 'firebase/auth';
import { auth } from '../config';
import { createUserDocument } from '../firestore';

// Kayıt
export async function registerWithEmail(
  email: string,
  password: string,
  username: string
) {
  const credential = await createUserWithEmailAndPassword(auth, email, password);

  // Displayname güncelle
  await updateProfile(credential.user, { displayName: username });

  // Firestore'da kullanıcı dokümanı oluştur
  await createUserDocument(credential.user.uid, { email, username });

  return credential.user;
}

// Giriş
export async function loginWithEmail(email: string, password: string) {
  const credential = await signInWithEmailAndPassword(auth, email, password);
  return credential.user;
}

// Çıkış
export async function logout() {
  await signOut(auth);
}

// Şifre sıfırlama
export async function resetPassword(email: string) {
  await sendPasswordResetEmail(auth, email);
}
```

3.3 Google Sign-In

typescript

```
// lib/firebase/googleAuth.ts
import { GoogleSignin } from '@react-native-google-signin/google-signin';
import { GoogleAuthProvider, signInWithCredential } from 'firebase/auth';
import { auth } from './config';
import { createUserDocument } from './firestore';

// App başlarken bir kere çağır (App.tsx veya _layout.tsx)
export function configureGoogle() {
  GoogleSignin.configure({
    webClientId: process.env.EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID,
    iosClientId: process.env.EXPO_PUBLIC_GOOGLE_IOS_CLIENT_ID,
  });
}

export async function signInWithGoogle() {
  await GoogleSignin.hasPlayServices();
  const { idToken } = await GoogleSignin.signIn();

  const credential = GoogleAuthProvider.credential(idToken);
  const result = await signInWithCredential(auth, credential);

  // Yeni kullanıcıysa Firestore'a ekle
  if (result.user.metadata.creationTime === result.user.metadata.lastSignInTime)
    await createUserDocument(result.user.uid, {
      email: result.user.email,
      username: result.user.displayName,
      photoURL: result.user.photoURL,
    });
}

return result.user;
}
```

3.4 Apple Sign-In

typescript

```
// lib/firebase/appleAuth.ts
import * as AppleAuthentication from 'expo-apple-authentication';
import { OAuthProvider, signInWithCredential } from 'firebase/auth';
import { auth } from './config';

export async function signInWithApple() {
  const credential = await AppleAuthentication.signInAsync({
    requestedScopes: [
      AppleAuthentication.AppleAuthenticationScope.FULL_NAME,
      AppleAuthentication.AppleAuthenticationScope.EMAIL,
    ],
  });

  const provider = new OAuthProvider('apple.com');
  const oauthCredential = provider.credential({
    idToken: credential.identityToken!,
    rawNonce: credential.authorizationCode!,
  });

  const result = await signInWithCredential(auth, oauthCredential);
  return result.user;
}
```

4. FIRESTORE VERİTABANI

4.1 Veri Modeli (Koleksiyonlar)

```
users/                                ← Kullanıcı profilleri
  {userId}/
    username: string
    email: string
    photoURL: string
    createdAt: timestamp
    onboardingCompleted: boolean
    profile/                          ← Biyometrik veriler (subcollection)
      biometrics: {
        gender: string
        age: number
        heightCm: number
      }
```

```

    gender: string, age: number,
    currentWeightKg, targetWeightKg
  }
  goals: string[]
  activityLevel: string
  challenges: string[]
  identity: string[]
stats/                                ← Hesaplanan değerler
  daily: {
    calories, protein, carbs, fat, water
  }
  estimated: {
    weeks, targetDate
  }

posts/                                ← Paylaşımlar (feed)
  {postId}/
    userId: string
    username: string
    content: string
    mediaURL: string | null
    likes: number
    createdAt: timestamp

channels/                              ← Uzman kanalları
  {channelId}/
    name: string
    ownerId: string
    members: string[]
    isExpert: boolean

```

4.2 Firestore Servisi

```

typescript

// lib/firebase/firestore.ts
import {
  doc,
  setDoc,
  getDoc,
  updateDoc,
  collection,
  addDoc,
  serverTimestamp,
  query,
  orderBy,
  limit,
  getDocs,

```

```
onSnapshot,  
} from 'firebase/firestore';  
import { db } from './config';  
  
// — KULLANICI —————  
  
// Yeni kullanıcı dökümanı oluştur  
export async function createUserDocument(  
  userId: string,  
  data: { email: string | null; username: string | null; photoURL?: string | null  
) {  
  await setDoc(doc(db, 'users', userId), {  
    ...data,  
    onboardingCompleted: false,  
    createdAt: serverTimestamp(),  
  });  
}  
  
// Kullanıcı profilini getir  
export async function getUserProfile(userId: string) {  
  const snap = await getDoc(doc(db, 'users', userId));  
  return snap.exists() ? snap.data() : null;  
}  
  
// Onboarding verilerini kaydet (tamamlandığında çağrılır)  
export async function saveOnboardingData(userId: string, onboardingData: {  
  goals: string[];  
  challenges: string[];  
  weightGoalKg: number;  
  timeline: string;  
  gender: string;  
  age: number;  
  heightCm: number;  
  currentWeightKg: number;  
  targetWeightKg: number;  
  activityLevel: string;  
  identity: string[];  
  channelChoice: string;  
  // Hesaplanan değerler  
  dailyCalories: number;  
  macros: { protein: number; carbs: number; fat: number };  
  estimatedWeeks: number;  
}) {  
  const userRef = doc(db, 'users', userId);  
  
  await updateDoc(userRef, {  
    onboardingCompleted: true,  
    'profile.goals': onboardingData.goals,
```



```

    'profile.challenges': onboardingData.challenges,
    'profile.activityLevel': onboardingData.activityLevel,
    'profile.identity': onboardingData.identity,
    'profile.channelChoice': onboardingData.channelChoice,
    'profile.biyometrics': {
      gender: onboardingData.gender,
      age: onboardingData.age,
      heightCm: onboardingData.heightCm,
      currentWeightKg: onboardingData.currentWeightKg,
      targetWeightKg: onboardingData.targetWeightKg,
    },
    'stats.daily': {
      calories: onboardingData.dailyCalories,
      ...onboardingData.macros,
    },
    'stats.estimated': {
      weeks: onboardingData.estimatedWeeks,
    },
  });
}

// — POSTS —————

// Post oluştur
export async function createPost(userId: string, content: string, mediaURL?: string) {
  const userSnap = await getDoc(doc(db, 'users', userId));
  const username = userSnap.data()?.username;

  await addDoc(collection(db, 'posts'), {
    userId,
    username,
    content,
    mediaURL: mediaURL || null,
    likes: 0,
    createdAt: serverTimestamp(),
  });
}

// Feed'i real-time dinle
export function subscribeFeed(callback: (posts: any[]) => void) {
  const q = query(
    collection(db, 'posts'),
    orderBy('createdAt', 'desc'),
    limit(20)
  );

  return onSnapshot(q, (snapshot) => {

```

```

    const posts = snapshot.docs.map(doc => ({
      id: doc.id,
      ...doc.data(),
    }));
    callback(posts);
  });
}

```

5. STORAGE (Fotoğraf/Video Upload)

5.1 Storage Servisi

typescript

```

// lib/firebase/storage.ts
import {
  ref,
  uploadBytesResumable,
  getDownloadURL,
  deleteObject,
} from 'firebase/storage';
import { storage } from './config';
import * as ImagePicker from 'expo-image-picker';
import * as VideoThumbnails from 'expo-video-thumbnails';

// Fotoğraf seç
export async function pickImage() {
  const result = await ImagePicker.launchImageLibraryAsync({
    mediaTypes: ImagePicker.MediaTypeOptions.All,
    allowsEditing: true,
    aspect: [1, 1],
    quality: 0.8,
  });

  if (!result.canceled) {
    return result.assets[0];
  }
  return null;
}

// Firebase'e upload (progress callback ile)
export async function uploadMedia(
  userId: string,
  uri: string,
  type: 'posts' | 'profile' | 'story',
  onProgress?: (progress: number) => void
) {

```

```

    ): Promise<string> {
      // URI'yi blob'a çevir
      const response = await fetch(uri);
      const blob = await response.blob();

      // Unique dosya adı
      const ext = uri.split('.').pop() || 'jpg';
      const filename = `${Date.now()}.${ext}`;
      const path = `${type}/${userId}/${filename}`;

      const storageRef = ref(storage, path);
      const uploadTask = uploadBytesResumable(storageRef, blob);

      return new Promise((resolve, reject) => {
        uploadTask.on(
          'state_changed',
          (snapshot) => {
            const progress = (snapshot.bytesTransferred / snapshot.totalBytes) * 100;
            onProgress?.(Math.round(progress));
          },
          (error) => reject(error),
          async () => {

            const downloadURL = await getDownloadURL(uploadTask.snapshot.ref);
            resolve(downloadURL);
          }
        );
      });
    }
  }
}

```

```

// Profil fotoğrafı güncelle
export async function updateProfilePhoto(userId: string, uri: string) {
  const url = await uploadMedia(userId, uri, 'profile');

```

```

  // Auth profil fotoğrafını güncelle
  const { updateProfile } = await import('firebase/auth');
  const { auth } = await import('./config');
  if (auth.currentUser) {
    await updateProfile(auth.currentUser, { photoURL: url });
  }

```

```

  // Firestore'u güncelle
  const { updateDoc, doc } = await import('firebase/firestore');
  const { db } = await import('./config');
  await updateDoc(doc(db, 'users', userId), { photoURL: url });

```

```

  return url;
}

```

```
// Medyayı sil
export async function deleteMedia(path: string) {
  const storageRef = ref(storage, path);
  await deleteObject(storageRef);
}
```

5.2 Upload Bileşeni (Örnek Kullanım)

typescript

```
// components/PostUploader.tsx
import { useState } from 'react';
import { View, Button, Image, Text, ActivityIndicator } from 'react-native';
import { pickImage, uploadMedia } from '@lib/firebase/storage';
import { createPost } from '@lib/firebase/firestore';
import { useAuth } from '@hooks/useAuth';

export function PostUploader() {
  const { user } = useAuth();
  const [selectedImage, setSelectedImage] = useState<string | null>(null);
  const [progress, setProgress] = useState(0);
  const [uploading, setUploading] = useState(false);

  const handlePick = async () => {
    const asset = await pickImage();
    if (asset) setSelectedImage(asset.uri);
  };

  const handlePost = async () => {
    if (!user || !selectedImage) return;
    setUploading(true);

    try {

```

```

const mediaURL = await uploadMedia(
  user.uid,
  selectedImage,
  'posts',
  setProgress
);
await createPost(user.uid, 'Bugünkü ilerleme! 💪', mediaURL);
setSelectedImage(null);
setProgress(0);
} finally {
  setUploading(false);
}
};

return (
  <View>
    {selectedImage && <Image source={{ uri: selectedImage }} style={{ width: 200, height: 200 }} />}
    {uploading && <Text>Yükleniyor: %{progress}</Text>}
    <Button title="Fotoğraf Seç" onPress={handlePick} />
    <Button title="Paylaş" onPress={handlePost} disabled={uploading} />
  </View>
);
}

```

🔒 6. FIRESTORE GÜVENLİK KURALLARI

Firebase Console → Firestore → Rules sekmesine yapıştır:

```

rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Kullanıcı kendi profilini okuyabilir/yazabilir
    match /users/{userId} {
      allow read: if request.auth != null;
      allow write: if request.auth.uid == userId;
    }

    // Postlar: herkes okuyabilir, sadece sahibi yazabilir
    match /posts/{postId} {
      allow read: if request.auth != null;
      allow create: if request.auth != null;
      allow update, delete: if request.auth.uid == resource.data.userId;
    }

    // Kanallar: üyeler okuyabilir

```

```
// channelId: string; ownerId: string;
match /channels/{channelId} {
  allow read: if request.auth != null &&
    request.auth.uid in resource.data.members;
  allow create: if request.auth != null;
  allow update: if request.auth.uid == resource.data.ownerId;
}
}
```



7. ONBOARDING AKIŞINA ENTEGRASYON

7.1 App Router — Auth Yönlendirme

typescript

```
// app/_layout.tsx
import { useEffect } from 'react';
import { useRouter, useSegments } from 'expo-router';
import { useAuth } from '@/hooks/useAuth';
import { getUserProfile } from '@/lib/firebase/firestore';

export default function RootLayout() {
  const { user, loading } = useAuth();
```

```

const router = useRouter();
const segments = useSegments();

useEffect(() => {
  if (loading) return;

  const inAuthGroup = segments[0] === '(auth)';
  const inOnboarding = segments[0] === 'onboarding';

  if (!user) {
    // Giriş yapmamışsa login'e yönlendir
    if (!inAuthGroup) router.replace('/(auth)/login');
  } else {
    // Onboarding tamamlandı mı kontrol et
    getUserProfile(user.uid).then((profile) => {
      if (!profile?.onboardingCompleted && !inOnboarding) {
        router.replace('/onboarding');
      } else if (profile?.onboardingCompleted && (inAuthGroup || inOnboarding))
        router.replace('/(tabs)');
      }
    });
  }
}, [user, loading]);

return <Slot />;
}

```

7.2 Onboarding Tamamlandığında Firebase'e Kaydet

typescript

```
// Onboarding'in son ekranında (result.tsx)
import { useOnboardingStore } from '@store/onboardingStore';
import { saveOnboardingData } from '@lib/firebase/firestore';
import { useAuth } from '@hooks/useAuth';

export default function ResultScreen() {
  const { user } = useAuth();
  const store = useOnboardingStore();
  const router = useRouter();

  const handleComplete = async () => {
    if (!user) return;

    await saveOnboardingData(user.uid, {
      goals: store.goals,
      challenges: store.challenges,
      weightGoalKg: store.weightGoalKg!,
      timeline: store.timeline,
      gender: store.gender!,
      age: store.age!,
      heightCm: store.heightCm!,
      currentWeightKg: store.currentWeightKg!,
      targetWeightKg: store.targetWeightKg!,
      activityLevel: store.activityLevel,
      identity: store.identityStatements,
      channelChoice: store.channelChoice!,
      dailyCalories: store.dailyCalories!,
      macros: store.macros!,
      estimatedWeeks: store.estimatedWeeks!,
    });

    router.replace('/(tabs)');
  };
};
```